

Deepfake and Manipulated Media Detection

A triage service for video, image and audio. Built to be auditable, and honest about what it does not know.

Scope	Deployment	Interface	Status
MVP, timeline reopened	Docker Compose, single GPU node	Web application	Pipeline verified end to end

1. Understanding the Problem

Fake and edited media is cheap to make now. Face swaps, puppeted video, cloned voices. It also spreads fast. On social platforms most of the damage happens in the first few hours, long before anyone can check a clip by hand. The task is to build something that helps find this content.

Reading past the surface of that, four things shape everything below.

1.1 The output is a signal, not a verdict

No detector is right all the time. Accuracy drops sharply on media the model was never trained on. A system that confidently says "FAKE" is more dangerous than useful. A false positive on real footage is itself a kind of misinformation, and it is the kind that gets a person blamed for something they did not do. So the honest output is a score, a band, and a clearly marked "not sure" range. Not a yes or no label.

1.2 Every upload is hostile

Anyone can upload anything. The file gets opened by media decoders, and those have a long history of memory safety bugs. On top of that, the scores themselves can be pushed around by an attacker who tweaks the input in small ways. Both of these have to be designed around. Neither can be assumed away.

1.3 The media is biometric, so keeping it is a risk

What gets uploaded here is faces and voices. Keeping that forever grows the damage of any future breach, and grows the legal exposure with it. So deletion has to be the default, and it has to be provable. But enough evidence has to survive that a disputed result can still be looked at again.

1.4 A result nobody can reconstruct is worthless

If someone challenges a decision weeks later, "the model said 84" is not good enough. You need to know which exact weights ran, how the per frame scores were combined, what settings were used, and a hash of the bytes that were analysed. All of it recorded at the time the decision was made. That record is part of the product, not just logging.

What this does not do

This system does not decide whether something is true, and it does not decide whether anything should be taken down. It looks for signs that media was synthesised or edited, and routes the result. Whether a video is authentic and whether it is honest are two different questions.

2. The Proposed Solution

A service that takes a video, image or audio file and returns three things. A calibrated score for how likely it is to be manipulated, a band that decides what happens next, and an audit record that makes the result reproducible. The underlying media is deleted on a timer by default.

2.1 Three pipelines, one place where decisions happen

Each type of media is handled by a pipeline suited to it. All three then meet at the same aggregation, the same band routing and the same retention logic. That keeps one place where a number turns into a decision, instead of three slightly different ones that drift apart over time.

2.2 Score the pieces, then combine them carefully

A model scores individual frames, faces or audio chunks. Those scores are then combined using a confidence weighted trimmed mean. Never a plain average, because a handful of badly cropped faces can drag a plain average anywhere they like. Items are also weighted down by how confident the face detection was, so a frame where the face was barely found counts for less.

2.3 Refuse to guess

If face detection finds nothing at all, the result is `undetermined`. It is never quietly turned into "authentic". If there is more than one face, each one is scored separately and the frame takes the worst of them. That way a single edited face in a crowd does not get averaged into nothing.

2.4 Bands, not raw percentages

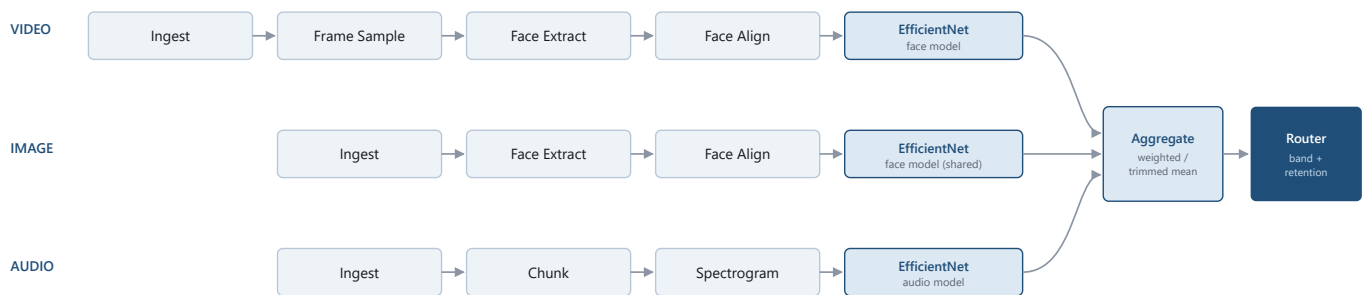
The combined score maps onto a band, and the band decides what happens. The bands cover the whole range with no gaps, so there is never an input the router does not know how to handle.

SCORE	BAND	WHAT HAPPENS
under 20	Likely authentic	Cleared automatically, deleted on schedule
20 to 40	Leaning authentic	Cleared automatically, deleted on schedule
40 to 60	Uncertain	Flagged for review and an alert is sent
60 to 80	Leaning manipulated	Flagged and alerted at low urgency. Still deleted on schedule
over 80	Likely manipulated	Flagged and alerted. Extended retention window opens
none	Undetermined	No usable signal. Flagged, and never scored as real or fake

The 60 to 80 range gets special care. It sits right below the most serious threshold, and the calibration around that threshold is not trusted enough to show people a raw percentage. So results there are logged and flagged rather than quietly passed through to normal deletion.

3. How the System is Built

3.1 The detection pipelines

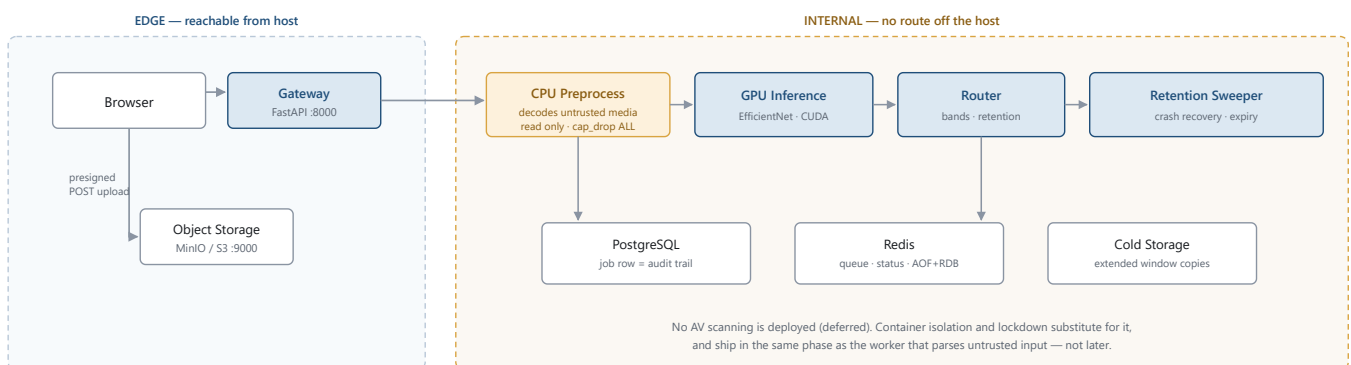


Face model is shared by video and image (one model_version_id). Audio is a separate model with its own.
For images, aggregation is the identity function — there is a single item to score.

Figure 1. Three pipelines that meet at shared aggregation and routing.

3.2 Services, and the trust boundary between them

The network layout here is a security control, not tidiness. The container that opens attacker supplied media has no route off the machine at all.



No AV scanning is deployed (deferred). Container isolation and lockdown substitute for it, and ship in the same phase as the worker that parses untrusted input — not later.

Figure 2. Two zones. Only the gateway and object storage can be reached from outside.

3.3 What happens to a request

1. The client asks for a job. The gateway rate limits the request, creates the database row, and hands back a signed upload grant.
2. The client uploads the file straight to object storage. Large media never goes through the API at all.
3. The client tells the gateway it is done, and the job is queued. Calling this twice is safe and does not queue the job twice.
4. The CPU worker decodes the file, samples frames, and finds and aligns faces. For audio it cuts the file into chunks and turns them into spectrograms.
5. The GPU worker scores every item and writes a row for each one.
6. The router combines the scores, picks a band, raises any flags, and starts the deletion of the working media.
7. Status updates are pushed over a WebSocket the whole time, with polling as a fallback if the connection drops.

4. Key Features

4.1 Deletion you can actually check TIER 1

Uploads and the face crops made from them are deleted once inference finishes. Every path that can delete something checks a hold flag first. That includes the normal delete, the recovery sweep that catches jobs a crashed worker left behind, and the cold storage timer. The flag column and the check that reads it were written in the same commit as the delete code, before anything could set the flag.

4.2 The job row is the audit trail TIER 1

Every result stores a SHA 256 hash of the bytes that were analysed, the id of the model that produced the scores, and the exact method and settings used to combine them. The per item scores are kept after the media is deleted, because they are just numbers rather than biometric data. So a disputed result can still be examined without keeping anyone face on disk.

4.3 An upload path that cannot be used as free storage TIER 1

Uploads go straight to storage by design, which means the API never sees those bytes and cannot check their size. So the upload grant carries a signed size limit that object storage enforces itself. The simpler kind of signed URL cannot express a size limit at all, which would have left the bucket open to unlimited writes.

4.4 The extended retention window TIER 2

When a result lands in the highest band, the face crops that actually drove that score are copied into cold storage on a fixed 30 day timer. Not the whole original file. That way a dispute has the specific evidence to look at, while the upload itself is still deleted on the normal schedule.

This is not a legal hold

It is a fixed timer that expires by itself, and it is described that way everywhere in the code, in the API responses and in this document. A timer that quietly runs out in the middle of a dispute is a liability dressed up as a safeguard. Relying on it for a real dispute needs legal sign off, which this codebase does not attempt to claim it has.

4.5 Isolation instead of virus scanning TIER 3 SUBSTITUTE

The worker that opens uploaded media has no antivirus in front of it. Instead it runs on an internal only network with no route off the machine, a read only filesystem, every Linux capability dropped, no ability to gain new privileges, a temporary directory that cannot execute anything, and hard limits on memory and process count.

4.6 Failing in ways you can see

- **Dead letter queue.** A job retries a limited number of times, then stops and is recorded with the reason. It is never silently lost, and never retried forever.
- **Durable status.** Redis runs with persistence turned on, and the service refuses to start without it, so a restart cannot wipe out the status of jobs that are still running.
- **Recovery sweeps.** Background sweeps find jobs that finished, or failed, or were abandoned, and clean up any media they left behind. Every one of those sweeps respects the hold flag.

4.7 Every result says how far it can be trusted

Results carry plain warnings attached to them. One says the score can be pushed around by an attacker editing the input. Another says how trustworthy the model itself is. That second one fails safe on purpose. If the system cannot tell which model produced a score, it gives the strongest warning rather than staying quiet.

5. Technology Choices

LAYER	CHOICE	WHY
API	Python 3.11, FastAPI, Uvicorn	Async support for pushing live status over WebSockets, and typed request and response models.
Models	PyTorch, EfficientNet	Good accuracy for the compute cost. Separate models for faces and for audio, each calibrated on its own.
Calibration	Temperature scaling	Cheap, and only needs a held out set. It is a snapshot taken at launch, nothing more.
Database	PostgreSQL 16	Transactional audit trail. JSON columns for the aggregation settings. Partial indexes make the cleanup sweeps fast.
Queue and status	Redis 7 with consumer groups	One dependency covers both the work queue and live status. If a worker dies holding a job, another one can pick it up without waiting for a restart.
Object storage	MinIO locally, S3 when deployed	Same upload behaviour in both, so nothing changes between them. MinIO simply disappears from the setup when deployed.
Packaging	Docker and Docker Compose	One image per service, and the network isolation is written down rather than assumed.
GPU	NVIDIA CUDA through WSL2	Checked on an RTX 4060 with 8 GB before any model code was wired in.
Testing	pytest, plus live checks	157 automated tests, plus scripts that check the running system against a real database, real Redis and a real bucket.
Supply chain	gitleaks	A secrets scan over the full history runs before anything is pushed publicly.

Why not Kubernetes yet

Setting up a cluster, secrets, ingress and storage volumes is days of work on its own, and on a single GPU machine it buys nothing that Compose does not already do. Compose runs the same set of services, so moving later is a packaging change rather than a redesign. The reason to move would be a second machine, a real uptime requirement, or needing to scale the GPU workers. Having spare time is not by itself a reason.

6. Implementation Plan

Harder work is deferred in clearly named tiers. The point of naming them is that nothing gets skipped quietly. Something deferred is written down as deferred, not left out and forgotten.

The phases below are what was actually built, in order. The original five day deadline has since been lifted, so the tiers now describe what depends on what rather than what fits in a week. Some things were only postponed because of time and are now open. Others are waiting on something real, like trained weights or legal sign off, and having more time does not change those at all.

PHASE	DELIVERED	CONTENTS
Day 1	Skeleton and contracts	Database schema including the hold flag, config, storage layer, queue and retry behaviour, rate limiting.
Day 2	Ingest and isolation	Signed uploads, API endpoints, live status. The CPU worker shipped together with its container lockdown, not after it.
Day 3	Scoring and routing	Inference workers behind a swappable backend, score combining, worst case rollup for multiple faces, band routing, review flags.
Day 4	Retention	Timed deletion, the hold flag check on every delete path, the extended retention window, and recovery sweeps.
Day 5	Proof it works	The whole stack running together, checks against a real database and real storage, and GPU access verified.

6.1 Where it stands

- ✓ The full stack runs end to end and every live check passes
- ✓ 157 automated tests passing, plus 60 or so checks against the running system
- ✓ Deletion confirmed by looking in the bucket, not by trusting a flag in the database
- ✓ The hold flag was proven to actually block deletion, on every path that can delete
- ✓ The audit trail is checked by reading the database columns back, not by trusting the API response
- ✓ The upload size limit is enforced by storage itself, confirmed by trying to break it
- ✓ One item can only ever produce one score row, enforced by the database
- ✓ Each result is tied to the model that actually produced it. If two models were involved, the job is refused rather than guessed at
- ✓ Every result states how far its model can be trusted, and that check fails safe
- Trained weights are not loaded yet. A placeholder scorer is active by default

6.2 What comes next

- Load real face and audio weights, then run the calibration step separately for each model.
- Check the score bands against real score distributions. Right now they sit on placeholder output.
- Replace the review flag with something a reviewer can actually use, once there is a reviewer.
- The adversarial input classifier. This is the biggest known gap and it is deliberately not half built.

6.3 Limitations, stated plainly

What this system does not claim

- **It is not robust against attackers.** Someone who knows what they are doing can edit an input to move the score. This is written down rather than hidden, and every result says so.
- **There is no legal hold.** The extended retention window is a fixed timer that expires on its own.
- **The calibration is not production validated.** It is a snapshot taken at launch and nothing keeps it up to date.
- **No compliance claim is made.** Scoped deletion and a partial audit trail are much better than nothing, but whether that meets any particular regulation is a legal judgement this codebase does not get to make.
- **It is not a detector yet.** The default backend derives a score from a hash of the input. It exists so the pipeline could be built and proven before real weights arrived. It carries no detection meaning, and it says so on every result it produces.

That last point is the one most worth being clear about. What has been proven so far is that the *system* works from end to end. Ingest, scoring, combining, routing, deletion and the audit trail. What has not been proven is *detection accuracy*, because that depends on weights that are not loaded yet. Treating those two as the same thing would be the most misleading thing this document could do.